

Methodology Quality Score (MQS)

Formal Specification, SPEC_VERSION = 1.0.0

Theseus / Noosphere

2026-05-14

Abstract

The Methodology Quality Score is the operational form of the five working criteria from `THE_META_METHOD.md`, attached to every Conclusion that carries at least one MethodologyProfile. This is the first *formal* version of the specification: it states the closed form of each of the five sub-scores, the exact gating function, the exact composite operator, and the boundary cases. Equations are numbered and the body refers to them. The specification is the source of truth for `noosphere/noosphere/evaluation/mqs.py`; `scripts/check_mqs_doc_consistency.py` fails CI if the two drift. The Markdown rendering at `docs/methods/MQS_Specification.md` is the same specification in prose; this PDF restates it in standard mathematical notation. No equation in this specification calls an LLM judge — the judge produces sub-score *evidence* and two numeric inputs, and the formulas are arithmetic on numbers.

1 Notation

A conclusion c carries text, a rationale, a topic hint, and links to forecasts, dissent claims, and a domain bound. $M(c)$ is the set of MethodologyProfiles attached to c (possibly empty); $m \in M(c)$ is one method. The five sub-scores and the composite are all in $[0, 1]$:

$$s_P, s_S, s_{AMF}, s_C, s_{DS} \in [0, 1], \quad \text{MQS}(c) \in [0, 1].$$

s_P is progressivity, s_S severity, s_{AMF} aim–method fit, s_C compressibility, s_{DS} domain sensitivity.

The typed inputs read from c and $M(c)$:

- $F(c) \in \mathbb{N}$ — forecast count linked to c .
- $B(c) \in \{0, 1\}$ — 1 iff c carries a future check-back date.
- $R(c) \in \mathbb{N}$ — decision-rule phrase count: regex matches of the decision-rule pattern in the conclusion text plus rationale.
- $\text{FM}(c) = \bigcup_{m \in M(c)} \text{failure_modes}(m)$ — concatenated failure-mode list; $A(c) = \bigcup_{m \in M(c)} \text{assumptions}(m)$, with $n = |A(c)|$.
- $D(c) \in \mathbb{N}$ — dissent-claim count.
- $j_S, j_{DS} \in [0, 1] \cup \{\perp\}$ — the LLM judge’s numeric scores for severity and domain sensitivity; $\perp$ means the judge produced no number.
- $\kappa \in [0, 1] \cup \{\perp\}$ — severity track-record ceiling.
- $\delta, \omega \in (0, 1]$ — severity drift penalty and objection penalty multipliers.

- $v \in \{\text{in_bounds}, \text{edge_case}, \text{out_of_bounds}\} \cup \{\perp\}$ — the domain-bound verdict.

$\text{clamp}(x)$ maps x into $[0, 1]$ and maps NaN to 0. Every formula in Section 2 is *total* — defined for every input, including $M(c) = \emptyset$ (Section 4).

2 Sub-scores

2.1 Progressivity s_P — deterministic

Did the analysis produce a prediction, implication, or decision rule that can later be checked? s_P is a piecewise constant of $F(c), B(c), R(c)$; the first matching clause wins:

$$s_P = \begin{cases} 1.00 & B(c) = 1 \wedge F(c) \geq 2 \wedge R(c) \geq 1, \\ 0.85 & (B(c) = 1 \wedge F(c) \geq 1) \vee F(c) \geq 2, \\ 0.65 & B(c) = 1 \vee F(c) \geq 1, \\ 0.40 & R(c) \geq 1, \\ 0.00 & \text{otherwise.} \end{cases} \quad (1)$$

2.2 Severity s_S — deterministic floor, judged, then deterministic caps

Would the procedure that produced c have caught the claim if it were false? s_S is computed in five stages, each of which can only move the score toward caution. The deterministic floor:

$$\text{floor} = \min(1, \text{SEVERITY_FAILURE_MODE_FLOOR_STEP} \cdot |\text{FM}(c)| + \text{SEVERITY_DISSENT_FLOOR_STEP} \cdot D(c)). \quad (2)$$

The judge, floored (with $j_S = \perp$ treated as 0):

$$s_S^{(0)} = \max(\text{floor}, \text{clamp}(j_S)). \quad (3)$$

The no-failure-modes cap — a method that lists no way it could fail is by construction not severe:

$$s_S^{(1)} = \begin{cases} \min(s_S^{(0)}, \text{SEVERITY_NO_FAILURE_MODES_CAP}) & \text{FM}(c) = \emptyset \wedge D(c) = 0, \\ s_S^{(0)} & \text{otherwise.} \end{cases} \quad (4)$$

The track-record ceiling — a thin or poorly calibrated `MethodTrackRecord` cannot project high severity onto new conclusions:

$$s_S^{(2)} = \begin{cases} \min(s_S^{(1)}, \text{clamp}(\kappa)) & \kappa \neq \perp, \\ s_S^{(1)} & \kappa = \perp. \end{cases} \quad (5)$$

Finally the multiplicative penalties — drift δ before objection ω , both in $(0, 1]$:

$$s_S = s_S^{(2)} \cdot \delta \cdot \omega. \quad (6)$$

Drift and objection penalties touch only severity: a method's recent calibration and its open objections bound how far to trust *this* claim of having weighed counter-evidence; they do not touch the other criteria.

2.3 Aim–method fit s_{AMF} — deterministic

Is the method actually capable of answering the question being asked? Since prompt 31 this sub-score is deterministic: it is the five-level rubric in `noosphere/noosphere/inquiry/aim_method_fit.py`, driven by the question typology. Let $\ell \in \{0, 1, 2, 3, 4\}$ be the rubric level assigned to c . Then

$$s_{\text{AMF}} = \ell/4 \in \{0.00, 0.25, 0.50, 0.75, 1.00\}. \quad (7)$$

The rubric and its worked examples are specified in `docs/methods/Aim_Method_Fit_Rubric.md`. Aim–method fit asks whether the method’s *output shape* matches the question; domain sensitivity (Section 2.5) asks whether the conclusion is inside the method’s *domain*. The two are distinct: a method can be in-domain and still misfit the question.

2.4 Compressibility s_{C} — deterministic, LLM-pruned

How many independent assumptions must hold for c to survive? Let $n = |A(c)|$ and let $d \in [0, n]$ be the count of assumptions the LLM judge classifies as decorative. The effective assumption count cannot fall below 1:

$$n' = \max(1, n - d). \quad (8)$$

The score decays with each load-bearing assumption beyond the first:

$$s_{\text{C}} = \frac{1}{1 + (n' - 1) \cdot \text{COMPRESSIBILITY_PENALTY_STEP}}. \quad (9)$$

With `COMPRESSIBILITY_PENALTY_STEP` = 0.25: $n' \in \{0, 1\} \mapsto 1.00$, $2 \mapsto 0.80$, $3 \mapsto 0.67$, $4 \mapsto 0.57$, $5 \mapsto 0.50$. The LLM only ever reduces n ; it cannot lift s_{C} above what the raw assumption count allows.

2.5 Domain sensitivity s_{DS} — judged, with deterministic verdict override; the gate

Where should this method stop being trusted, and is c inside or outside that domain? Define the pre-verdict score from the judge and a deterministic floor:

$$s_{\text{DS}}^0 = \max(\text{base}, \text{floor}), \quad \text{base} = \begin{cases} \text{clamp}(j_{\text{DS}}) & j_{\text{DS}} \neq \perp, \\ \text{DOMAIN_BACKFILL_DEFAULT} & j_{\text{DS}} = \perp, \end{cases} \quad \text{floor} = \begin{cases} \text{DOMAIN_NO_FAILURE_MODES_FLOOR} & \text{DOMAIN_NO_FAILURE_MODES} \\ 0 & \end{cases} \quad (10)$$

The domain-bound verdict v then overrides:

$$s_{\text{DS}} = \begin{cases} 0 & v = \text{out_of_bounds}, \\ \min(s_{\text{DS}}^0, \text{EDGE_CASE_DOMAIN_CEILING}) & v = \text{edge_case}, \\ s_{\text{DS}}^0 & \text{otherwise.} \end{cases} \quad (11)$$

An `out_of_bounds` verdict is a hard zero — the orchestrator has established c is outside the method’s declared `DomainBound`. The deterministic floor `DOMAIN_NO_FAILURE_MODES_FLOOR` = 0.10 keeps a method with no declared failure modes off zero — its domain claim is *unverifiable*, not *failed* — but, crucially, that floor is below the gate threshold θ (Section 3), so the floor alone never opens the gate.

3 Composite

Domain sensitivity is a *gate*, not a weighted addend and — since this formal version — not a soft multiplicative penalty either. With gate threshold $\theta = \text{DS_GATE_THRESHOLD} = 0.15$:

$$\text{MQS}(c) = \begin{cases} 0 & s_{\text{DS}} < \theta, \\ \text{wgm}(s_{\text{P}}, s_{\text{S}}, s_{\text{AMF}}, s_{\text{C}}) & s_{\text{DS}} \geq \theta. \end{cases} \quad (12)$$

The gate is closed on the passing side: $s_{\text{DS}} = \theta$ exactly opens it. The combining operator is the weighted geometric mean, with the `SUBSCORE_WEIGHTS` as exponents:

$$\text{wgm}(s_{\text{P}}, s_{\text{S}}, s_{\text{AMF}}, s_{\text{C}}) = s_{\text{P}}^{w_{\text{P}}} \cdot s_{\text{S}}^{w_{\text{S}}} \cdot s_{\text{AMF}}^{w_{\text{AMF}}} \cdot s_{\text{C}}^{w_{\text{C}}}, \quad \sum w_i = 1. \quad (13)$$

The four weights are equal at 0.25, so $\text{wgm} = (s_{\text{P}} s_{\text{S}} s_{\text{AMF}} s_{\text{C}})^{1/4}$. The operator has the weakest-link property:

$$(\exists i : x_i = 0) \implies \text{wgm}(x) = 0. \quad (14)$$

It is monotone non-decreasing in each argument, equals 1 if and only if every argument equals 1, and stays in $[0, 1]$.

3.1 Why this composite

The Round 17 prose draft used $\text{MQS}(c) = s_{\text{DS}} \cdot \text{mean}(s_{\text{P}}, s_{\text{S}}, s_{\text{AMF}}, s_{\text{C}})$ — a soft penalty in which $s_{\text{DS}} = 0.5$ merely capped the composite at 0.5. The firm’s position on a method pointed outside its domain is stronger than a discount: such a method is not producing a worse answer, it is producing one the firm has no basis to trust. Hence the hard gate of Eq. (12). Among the three operators that share the weakest-link property of Eq. (14) — the minimum, the harmonic mean, and the weighted geometric mean — the minimum discards three of four signals and the harmonic mean is so dominated by the smallest input that the others barely register. The weighted geometric mean keeps all four with proportional weight, still refuses redemption, and is the exact multiplicative analogue of the prose draft’s arithmetic mean (the same weight map, used as exponents). It is the choice.

4 Boundary cases

Each rule below is pinned by a named test in `noosphere/tests/test_mqs_spec.py`.

Insufficient data. No sub-score is ever undefined. With $M(c) = \emptyset$: s_{P} is still computed from c ; Eq. (2) gives $\text{floor} = 0$ and Eq. (4) caps s_{S} at 0.35; $s_{\text{AMF}} = 0.5$ (rubric level 2, “cannot verify fit”); Eq. (8)–(9) give $n' = 1$ and $s_{\text{C}} = 1.0$; Eq. (10) floors s_{DS} at `DOMAIN_NO_FAILURE_MODES_FLOOR` and, with no judge, uses `base = DOMAIN_BACKFILL_DEFAULT`. Insufficient data resolves to a defined, conservative number.

At the gate threshold. The gate condition $s_{\text{DS}} < \theta$ is strict, so $s_{\text{DS}} = \theta$ opens the gate and $s_{\text{DS}} = \theta - \varepsilon$ closes it. The composite is discontinuous at θ : this is intentional — it makes the gate a verdict, not a dial. Note `DOMAIN_NO_FAILURE_MODES_FLOOR` = 0.10 < θ = 0.15.

Inputs disagree. When a deterministic signal and the LLM judge disagree, the deterministic signal wins, toward caution: an `out_of_bounds` verdict forces $s_{DS} = 0$ over any j_{DS} (Eq. (11)); the no-failure-modes cap overrides a high j_S (Eq. (4)); the deterministic floor overrides a low j_S via the max of Eq. (3); the track-record and edge-case ceilings override high judge scores via min. The judge moves a sub-score only within the window the deterministic rules leave open.

A non-gate sub-score is exactly 0. If any of s_P, s_S, s_{AMF}, s_C is 0 and the gate is open, Eq. (14) gives $MQS(c) = 0$. This is intentional: a conclusion that produces nothing checkable, or whose method cannot answer the question at all, has a failing methodology quality regardless of the other axes.

Out-of-range or NaN inputs. Every input passes through clamp before any arithmetic; clamp confines values to $[0, 1]$ and maps NaN to 0. The composite is always a real number in $[0, 1]$.

5 Composite tiers

The composite is continuous in $[0, 1]$, but the firm reasons about it in four tiers, each an inclusive lower bound:

Tier	Composite $\geq$
strong	0.66
adequate	0.40
provisional	0.15
failing	0.00

`tier_rank` gives each tier an ordinal (`failing` = 0, ..., `strong` = 3). A re-score that drops a conclusion a tier is routed to the founder’s queue rather than silently downgraded.

6 Constants registry

Every constant the specification depends on, mirrored exactly by `MQS_CONSTANTS` in the code and checked bidirectionally by `scripts/check_mqs_doc_consistency.py`.

Constant	Value	Role
<code>SPEC_VERSION</code>	1.0.0	version of this specification.
<code>MQS_SCHEMA</code>	<code>theseus.mqs.v1</code>	schema string on every MQS row.
<code>PROMPT_VERSION</code>	<code>mqs-prompt-v2.0</code>	LLM-judge prompt version.
<code>DS_GATE_THRESHOLD</code>	0.15	gate threshold θ (Eq. (12)).
<code>COMPOSITE_OPERATOR</code>	<code>weighted_geometric_mean</code>	combining operator (Eq. (13)).
<code>SEVERITY_FAILURE_MODE_FLOOR_STEP</code>	0.15	per-failure-mode floor step (Eq. (2)).
<code>SEVERITY_DISSENT_FLOOR_STEP</code>	0.10	per-dissent floor step (Eq. (2)).
<code>SEVERITY_NO_FAILURE_MODES_CAP</code>	0.35	cap when no failure modes (Eq. (4)).
<code>COMPRESSIBILITY_PENALTY_STEP</code>	0.25	per-assumption penalty step (Eq. (9)).
<code>DOMAIN_NO_FAILURE_MODES_FLOOR</code>	0.10	s_{DS} floor when no failure modes (Eq. (10)).
<code>DOMAIN_BACKFILL_DEFAULT</code>	0.5	s_{DS} base when no judge (Eq. (10)).
<code>EDGE_CASE_DOMAIN_CEILING</code>	0.4	s_{DS} cap under <code>edge_case</code> (Eq. (11)).

Constant	Value	Role
EVIDENCE_STR_CAP	600	per-string evidence-blob character cap.

The four SUBSCORE_WEIGHTS (all 0.25) and the COMPOSITE_TIERS lower bounds are checked separately by the same script.

7 Versioning and changelog

SPEC_VERSION versions this specification; the first formal version is 1.0.0. Any material change to a formula, a constant, a boundary rule, or the composite operator bumps SPEC_VERSION and adds a changelog row. PROMPT_VERSION versions the LLM-judge prompt and is independent.

SPEC_VERSION	Date	Change
1.0.0	2026-05-14	First formal specification. The composite changed from the Round 17 prose draft’s soft multiplicative gate, $s_{DS} \cdot \text{mean}(s_P, s_S, s_{AMF}, s_C)$, to the hard piecewise gate of Eq. (12) at $\theta = 0.15$ followed by the weighted geometric mean of Eq. (13). Both the code (<code>composite_score</code>) and the spec were changed together — they had previously agreed on the soft formula. Rationale in Section 3. Every previously-magic constant was named and the constants registry added.